MagniData: Opening the Black Box After Every Epoch

Evidence-driven training and dataset curation for computer vision

Asser AboElkhair and Reinier Oves
Precision AI {asser, reinier}@precision.ai

github.com/Precision-AI-Inc/magnidata October 2026

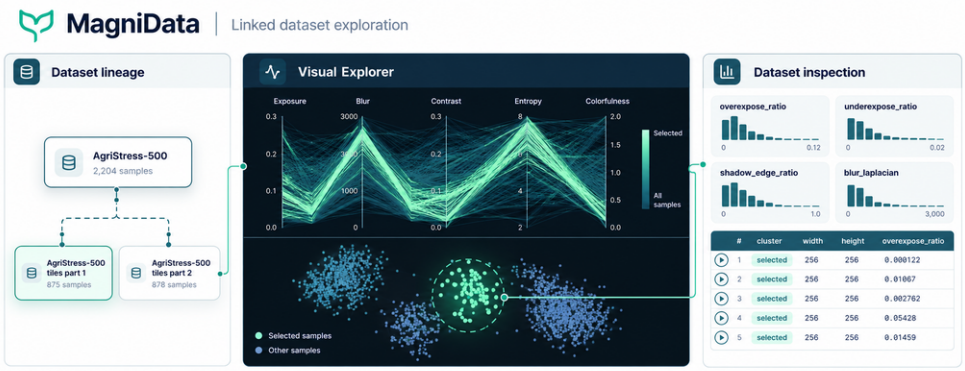


Figure 1: MagniData links the dataset's lineage (left), a visual explorer where one brush selects samples across parallel coordinates and a 3D view (centre), and the inspection of the selection (right). Illustration of the interface.

Abstract. Computer vision models are routinely treated as black boxes: a training run ends in an aggregate metric, and when that metric falls short of expectations the customary response is to increase model capacity, to train for more epochs, or to collect more data. Such decisions rest on convention rather than on evidence about the data, although a model may equally fail because of the quality, coverage or annotation of the samples it is given. We present MagniData, a visual analysis tool that relates the content of a dataset to the learning behaviour of a model. Per-image training signals recorded at every epoch are placed in a common space with photometric, geometric and embedding features, which makes it possible to identify persistently difficult samples, to select their neighbours in feature space by interactive brushing, and to narrow down the cause of failure to model capacity, under-representation or data quality. Derived datasets, merges and complements are recorded in a lineage graph that links each dataset to the experiments trained on it. Because data accumulates faster than it can be annotated, the same signals are also used to rank candidate images from a global pool by their similarity to the samples that are hard for the model, with the aim of directing human annotation and review towards the data most informative for learning. We describe the interaction design and experiments with four models on an agricultural segmentation dataset; quantifying the reduction in annotation effort remains future work.

1 Introduction

Dataset inspection has been a bottleneck in computer vision for decades, and the volume of data now makes it harder than ever to inspect by eye. Industry estimates put the number of photographs taken each year at roughly two trillion [1], which at typical smartphone file sizes amounts to exabytes of new data annually (our own order-of-magnitude estimate). Vision models have grown into this abundance. Vision transformers were scaled on JFT-3B, a proprietary set of about three billion images [2]; LAION-5B provides 5.85 billion web image–text pairs [3]; DataComp starts from a pool of 12.8 billion [4]; and the self-supervised DINOv3 was trained on roughly 1.7 billion images [5]. At a few hundred kilobytes per web image, a corpus of this size already occupies hundreds of terabytes. Yet size is not what separates the strongest results. DINOv2 relied on 142 million images selected by an automatic retrieval pipeline [6], clustering-based curation can match or surpass manual curation [7], and the 1.4-billion-sample DataComp-1B subset outperforms the larger LAION-2B [4]. What the model sees matters at least as much as how much of it there is, which is the premise of data-centric AI [8].

Annotation, where a task requires it, is the bottleneck. Dense labels such as segmentation masks are slow and expensive to produce by hand, and even Segment Anything built its billion-mask dataset largely with a model in the loop [9]. Commercial annotation services and open platforms such as CVAT and Label Studio organise this effort, but they do not say *which* images are worth labeling. Labels are also imperfect, and the errors that remain shape what a model can learn [10].

A mature set of tools surrounds this problem, each covering one stage. DVC versions data and pipelines [11]; MLflow tracks the parameters and metrics of every run [12]; and FiftyOne lets users browse images, labels and predictions, visualise embeddings, search by similarity and mine hard samples [13]. Research on training dynamics has shown that the loss history of an individual example exposes hard, ambiguous and mislabeled samples [14, 15, 16, 17]. The practical difficulty is that these views are disconnected. An experiment tracker knows that a run scored an mIoU of 0.71 but not which images account for the missing 0.29; a data browser can find look-alikes but does not know which of them the model fails on; a versioning tool records the bytes but not why they were

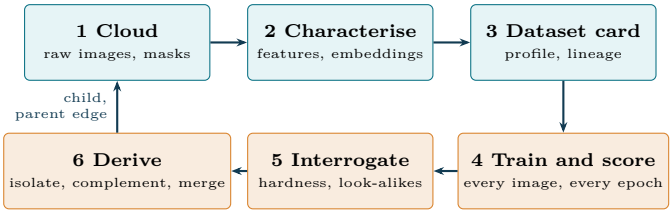


Figure 2: The procedure. Datasets (blue) are profiled and described once; experiments and curation (orange) then produce new datasets, each linked to its parents.

chosen. Teams must therefore decide what to label, what to add and what to trust without a shared picture of the data and of the model's behaviour on it.

We present MagniData, which closes this gap in two ways. First, it represents the dataset at a first glance through its *multiparametric* description, the photometric and geometric properties of every image, together with its embedding, so that the structure of the collection can be read before any model is trained. Second, it couples this representation to *continuous training*: after every epoch each image is scored and the result joins the same views. *Visual queries* then select more samples that fall under the same conditions as a sample of interest, and the training signal tells the user where the model is reliable and where it is not, so that risk can be mitigated, by curating or relabeling, or fenced, by documenting the conditions under which the model should not be trusted. The remainder of the paper describes the procedure that organises this work, and how a user moves from a failure to its look-alikes, to its cause, and to the next dataset.

2 MagniData Architecture

We propose to organise visual dataset analysis as a repeatable procedure rather than as a collection of tools. Its unit is the *dataset*: a named set of images together with a table of their properties. Every step takes a dataset as input and produces a dataset or an experiment as output, so that each iteration leaves a trace (Fig. 2).

From the cloud to a table of properties. The procedure starts from raw data, typically a very large collection of images with or without masks, which we call the *cloud*: the pool from which every other dataset is drawn. A preparation step runs once and reduces each image to a compact description: photometric properties (colour, exposure, sharpness, noise, compression, learned quality), geometry derived from the annotations where they exist (coverage, relative object area, number and overlap of instances), and, optionally, an embedding from a general-purpose encoder, together with precomputed three-dimensional projections of both. From then on the user works only with this table of features. The images themselves stay where they are and are retrieved on demand, with their masks and overlays, when a sample is opened for inspection. Interactive exploration therefore depends on the number of images and not on their resolution; on a 47,281-image dataset with 1024-dimensional embeddings, precomputing the projections reduced the first three-dimensional view from 27 s to 0.8 s.

Every dataset has a card and a place in the lineage. Each dataset is introduced by a description card (Fig. 3): its origin, the number of samples and properties, a small histogram of each property, example images, annotation



Figure 3: Lineage workspace and description card. AgriStress-500 → tiles part 1 → "Hard tiles" (173 samples; the yellow badge pulses while a run trains). The card shows the dataset's properties as histograms and tabs for images, annotations, runs and lineage.

coverage, the experiments trained on it, and its parents and children. The cards are the nodes of a lineage graph, and the edges record derivation.

Datasets are created by acting on evidence. Three operations create new datasets, and none modifies its parents. *Isolate*: a set of images on which a model struggles is selected and saved as a new dataset, for review, relabeling or a focused experiment. *Complement*: more images similar to the problematic ones are found in the cloud, as those nearest to the selection in feature space, so that new annotation effort is spent where the model is weak. *Combine*: several datasets are merged into a larger one, without duplicates, with every source recorded as a parent. Together they turn the lineage graph into a record of the reasoning behind each model.

A standard procedure. The procedure is therefore: collect raw data in the cloud, characterise it once, describe each dataset with a card, train and score every image at every epoch, interrogate the result for failures and their look-alikes, and derive a new dataset by isolating, complementing or combining before training again; each new run records the previous one as its parent, and a comparison on the dataset reports image by image whether the curated set stopped being hard. Because every step is expressed in terms of datasets and experiments, it is independent of the model and the trainer, and any two iterations can be compared.

3 Model in the Loop

A diagnostic instrument is convincing only if it works with more than one kind of model. MagniData therefore launches training from a single dialog for three model families, which differ in how much they learn from the images and how much from prior knowledge (Table 1). They are proofs of concept for the loop, not leaderboard entries: whatever its architecture, each model reports the loss of every image after every epoch, and the tool then treats all of them alike. Every model was trained for 100 epochs on the same 875 tiles of AgriStress-500 (a validation fraction of 20%, one seed, no tuning) on a single laptop GPU.

3.1 TinyUNet: the baseline that runs anywhere

TinyUNet is a compact encoder–decoder that learns the task from scratch, from the labelled images alone. It is deliberately modest, fast enough for a first smoke test on a new dataset. With no prior knowledge to lean on, its difficulties are those of the data and the task, and its many failures make the loop easy to see working. After

Table 1: The models of the loop, trained for 100 epochs on 875 tiles. Validation mIoU is reported at the last epoch, with the best epoch in brackets. AgriTune draws its own validation split, so its scores are not directly comparable with the first two rows.

Model	Learns from	Val. mIoU	Time
TinyUNet	images alone	0.37 (0.39)	17 min
LR-ASPP	ImageNet prior	0.71 (0.74)	17 min
MLP Probe	frozen embedding	0.60 (0.60)	40 min
TokenFPN	frozen embedding	0.53 (0.67)	38 min

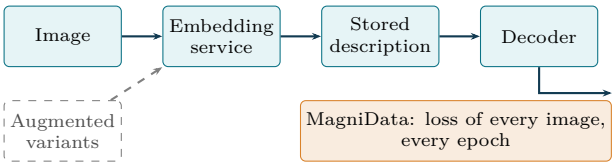


Figure 4: AgriTune learns a decoder on top of a general-purpose embedding. The service is called once per original image; augmented variants (dashed) would cost one call each and are left out.

100 epochs it reached a validation mIoU of 0.37.

3.2 LR-ASPP: a standard, efficient network

LR-ASPP pairs a lightweight MobileNetV3 backbone, pretrained on ImageNet, with a light segmentation head; it is a design intended for efficient deployment. It brings knowledge from generic images and is the strongest model here, reaching 0.71 (0.74 at its best epoch). It lets the user ask whether a better model fails on the same images.

3.3 AgriTune: learning from a general-purpose embedding

AgriTune, the third family, does not learn to see. It sends each original image once to a hosted embedding service, keeps the resulting description, and trains a lightweight decoder (an MLP Probe or a richer TokenFPN) to turn it into a segmentation mask (Fig. 4). The decoder thus inherits, through distillation, the broad agricultural knowledge of a general-purpose embedding, and the encoder is never changed. The price is explicit: augmenting an image would require a new request for every variant, so the cached recipe leaves augmentation out and the decoder does not learn the invariances it normally teaches. This limitation motivates Section 7. The MLP Probe reached 0.60. The TokenFPN peaked at 0.67 in epoch 79 but ended at 0.53, with a training mIoU of 0.87: it had begun to memorise the training tiles.

3.4 What the losses look like

Figure 5 shows what a user sees first. Validation accuracy separates the models clearly, and the MLP Probe's losses fall quickly and flatten, with the validation loss settling at about twice the training loss for cross-entropy (0.051 against 0.024) and one and a half times for Dice (0.082 against 0.057). This is a healthy but visible generalisation gap, and the curves cannot say *which* tiles make it up or whether the other models fail on them too. That is what the per-image columns derived from the same run are for.

3.5 Do the models fail on the same images?

Because every model scores every image at every epoch, the four runs can be compared image by image. Taking each tile's mean loss over training as its difficulty, the rank correlation between any two models lies between 0.75 and 0.87, even though their validation scores range from 0.37 to 0.71 and their learning signals are entirely different (Fig. 7). Of the 175 hardest tiles of each model (the worst fifth), between 48% and 78% are also among the hardest of another model, against 20% expected by chance, and 31% are among the hardest of all four. The overlap is highest between the three models that rely on the same kind of signal (TinyUNet, MLP Probe and TokenFPN, 71–78%) and lowest for the pretrained LR-ASPP (48–53%).

A large part of what is hard is hard for every model, as one expects if the difficulty lies in the images or their labels, and a bigger model will not remove it. A smaller part is model-specific, which is where capacity, prior knowledge or the representation can help: if a larger decoder removes the failures the limit was *capacity*; if only the end-to-end model does, the frozen embedding lacks the information (*representation*); a shared photometric signature among the survivors points to what augmentation would cover (*missing invariance*); and failures that persist in every run point to *labels or coverage*. The shared core is the natural first candidate for review, and its look-alikes (Section 5) show how far it extends.

Which of these causes applies can be read from the masks themselves. Both cases occur in the masks (Fig. 6). In the first the label is plausible and the prediction is not: a wilted leaf that the frozen-embedding decoders leave out, neighbouring classes that TinyUNet confuses and LR-ASPP resolves, and a thin stem that the embedding cannot resolve. In the second the predictions are plausible and the label is questionable: an angular polygon covering a leaf tip that occupies a thin strip of the tile, a dry leaf left unlabelled although the tile next to it labels a similar one, and a plant cut by the tile border with no label. These are candidates identified by eye, not verified errors, and they remain few among the hard tiles: most of them are model failures on small plants.

4 Interrogating the Model, Epoch by Epoch

After each scored epoch the trainer re-scores every image in eval mode, without augmentation, and logs one file of per-image loss terms. MagniData turns it into columns: the loss of each term, its mean over epochs, its slope, its percentile rank within the image's own split, and *hardness*, the share of scored epochs in which the image was among the worst 20% of its split. They sit on the same parallel-coordinates plot as colour, exposure, sharpness, noise, geometry and embedding features (Fig. 1), and an epoch scrubber replays how the failures move as training proceeds. Opening a sample shows its image, ground truth, the model's *prediction* and its own per-term loss curves. This is the moment accuracy becomes a human judgement: after seeing ten failures beside their labels, a user can say whether the model or the label is wrong, or the case is ambiguous, and whether the error is tolerable. Because the loss is split by term, a high Dice rank with a low cross-entropy rank means the pixels are mostly right but the region's *shape* is off, which points to a coarse or shifted mask rather than a class problem.

Hardness says that something is wrong, not who is at fault, so it is read against a second question: do independent runs agree? A tile that is hard for every model, where the models agree with one another but not with the label, is a *label suspect*; one where they disagree with each

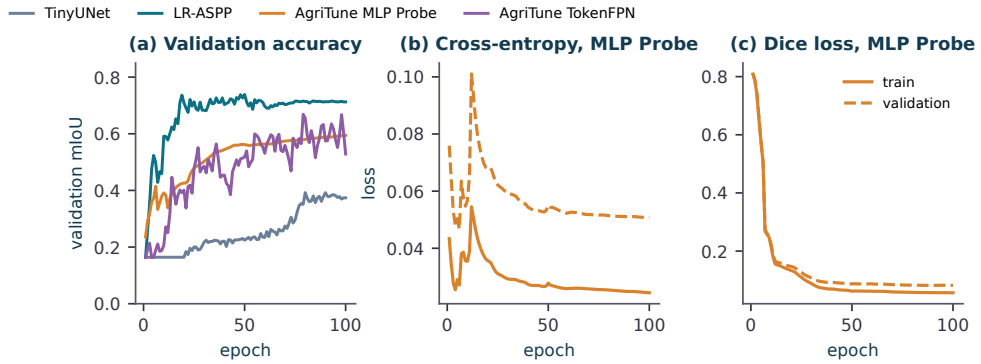


Figure 5: Aggregate behaviour of the four models over 100 epochs. (a) Validation mIoU. (b,c) Training (solid) and validation (dashed) cross-entropy and Dice loss of the AgriTune MLP Probe. The curves summarise the whole dataset; they do not say which images are responsible for the gap.

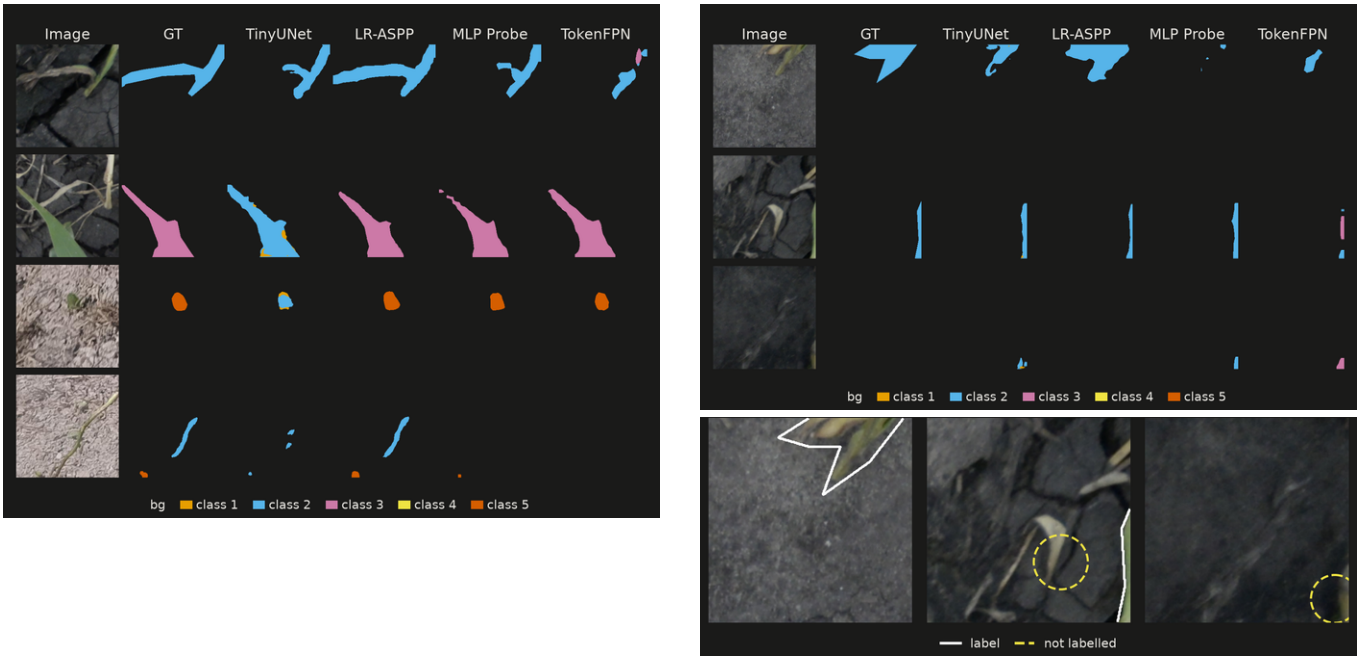


Figure 6: Hard tiles by probable cause, from the 256 px tiles of AgriStress-500. Each row shows the image, the label and the four models. (Left) The label is plausible and the prediction is bad: a missed wilted leaf, a class confusion and a thin stem. (Right) The prediction is acceptable and the label is questionable: an oversized polygon, an unlabelled dry leaf and an unlabelled plant at the tile border. The strip below enlarges these three tiles: the white outline is the label, and the yellow circles mark plants without one (the dry leaf sits next to a labelled plant).

other as well points to the model or its representation

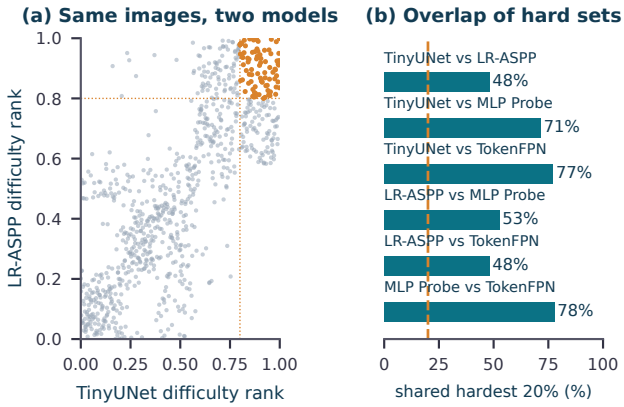


Figure 7: Do models agree on what is hard? (a) Difficulty rank of each of the 875 tiles under TinyUNet and LR-ASPP; orange marks tiles in the hardest fifth of both. (b) Share of the hardest fifth that two models have in common; the dashed line is the 20% expected by chance.

(Section 3.5). Only a model that did not fit a tile can testify about its label, since the others reproduce it: the TokenFPN reached a training mIoU of 0.87 against 0.53 on validation. Hardness is therefore always ranked within a tile's own split, and evidence about labels should come from held-out epochs.

5 Are They Alone? Pick the Similar Ones

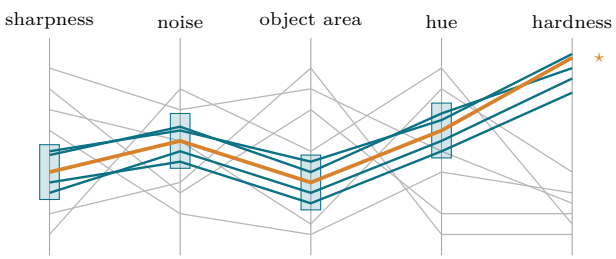


Figure 8: Schematic of the brush-by-similarity test. A hard sample (⋆, orange) fixes a narrow band on four feature axes; the selected look-alikes (teal) land high on `hardness`: a trend, not an accident.

The most useful question after seeing one failure is whether it is alone. Parallel coordinates [18] turn it into a gesture. Every image is one polyline; the preview of a hard sample gives its value on each feature, and the user brushes a narrow band around those values on a few axes (Fig. 8). The surviving polylines are the sample's look-alikes, and the 3D view, histograms and table update with them.

Two readings, two actions. If the look-alikes are also hard, the failure is a *tendency*: a region of feature space, say low sharpness with small object area, where the model fails as a group. It is measurable (counts and mean loss inside versus outside the band), and it points to coverage or quality rather than to individual labels. If the look-alikes are easy, the sample is an outlier of its own neighbourhood and its label is the first suspect. Widening the band until neighbour hardness falls back to the dataset average gives a rough extent of the failure region. The band is currently drawn by hand; a one-click version is planned.

Two cautions apply. First, difficulty depends on how much is labelled: in AgriStress-500 tiles without stress are almost never hard, so a band should be compared with look-alikes of similar labelled area and not with the whole dataset. Second, the tiles are cut from large mosaics, so a tile's spatial neighbours are look-alikes too. An unlabelled dry leaf beside a labelled withered one on the adjacent tile (Fig. 6) is an inconsistency that no single-image view reveals; brushing on tile coordinates would expose it.

6 Limitations

Hardness and ranks order attention: a persistently hard image is a candidate, not a verdict. Part of the cross-model agreement of Section 3.5 reflects that tiles with stress are harder than empty ground: on the 306 stressed tiles alone the rank correlations fall from 0.75–0.87 to 0.05–0.76, and the shared core is smaller than it first appears. The label problems shown in Fig. 6 were found by eye on a handful of tiles; only 38 of the 875 tiles are held out for all four models, because AgriTune draws a different validation split, so held-out evidence about labels is thin. Plant fragments cut by tile borders are common (47% of stressed tiles have every labelled object touching the border) but do not by themselves make a tile harder. The look-alike test is manual and its meaning depends on the axes the user chooses; the complement is a centroid-distance heuristic, not learned retrieval; and lineage records parent edges between dataset files, not content-addressed versions. The model comparison uses one seed, untuned settings and the mean loss over training. We have not yet measured label-error detection precision, or whether the cross-run reading predicts the benefit of scaling a model.

7 Future Work

Distillation from a frozen embedding is cheap because it never calls the API twice for the same image; that is also why it forgoes augmentation. We will therefore release a *contrastive-learning edition*, so that the invariances augmentation normally provides can be learned without a new API call per view, and MagniData can inspect the geometry of the resulting embedding in the same way it inspects segmentation loss today.

The limitations above suggest four nearer steps. (i) Make hardness independent of how much is labelled, by ranking within strata of labelled area and by adding a per-class IoU axis. (ii) Obtain a held-out loss for every tile with k-fold training, so that label-error detection, in the manner of confident learning [10], no longer depends on which tiles one split happened to hold out, and add a cross-run disagreement column. (iii) Flag inconsistent neighbours on the mosaic, and agree a rule for border fragments, such as overlapping tiles or dropping labelled objects below a minimum area. (iv) Have an agronomist review the classes that models swap, 2 and 4 in our data. We will evaluate hardness-based candidates against injected label errors, test the four-cause reading on deliberately corrupted datasets, repeat the comparison over several seeds, and add one-click similarity brushing and embedding-based complements.

References

[1] PetaPixel. The number of photos taken in 2025 is expected to exceed two trillion (industry estimate, Rise Above Research). June 2025.

[2] X. Zhai, A. Kolesnikov, N. Houlsby, L. Beyer. Scaling vision transformers. CVPR, 2022.

[3] C. Schuhmann et al. LAION-5B: an open large-scale dataset for training next generation image-text models. NeurIPS Datasets and Benchmarks, 2022.

[4] S. Y. Gadre et al. DataComp: in search of the next generation of multimodal datasets. NeurIPS Datasets and Benchmarks, 2023.

[5] O. Siméoni et al. DINOv3. arXiv:2508.10104, 2025.

[6] M. Oquab et al. DINOv2: learning robust visual features without supervision. TMLR, 2024.

[7] H. V. Vo et al. Automatic data curation for self-supervised learning: a clustering-based approach. TMLR, 2024.

[8] D. Zha et al. Data-centric artificial intelligence: a survey. ACM Computing Surveys, 2025.

[9] A. Kirillov et al. Segment anything. ICCV, 2023.

[10] C. Northcutt, L. Jiang, I. Chuang. Confident learning: estimating uncertainty in dataset labels. JAIR 70, 2021.

[11] Iterative.ai. DVC: Data Version Control. https://dvc.org.

[12] M. Zaharia et al. Accelerating the machine learning lifecycle with MLflow. IEEE Data Engineering Bulletin 41(4), 2018.

[13] Voxel51. FiftyOne: open-source tool for building high-quality datasets and computer vision models. 2020.

[14] S. Swayamdipta et al. Dataset cartography: mapping and diagnosing datasets with training dynamics. EMNLP, 2020.

[15] M. Toneva et al. An empirical study of example forgetting during deep neural network learning. ICLR, 2019.

[16] G. Pleiss et al. Identifying mislabeled data using the area under the margin ranking. NeurIPS, 2020.

[17] M. Paul, S. Ganguli, G. K. Dziugaite. Deep learning on a data diet: finding important examples early in training. NeurIPS, 2021.

[18] A. Inselberg. The plane with parallel coordinates. The Visual Computer 1(2), 1985.